

CSC 214: Algorithm & Complexity Analysis (2 Units)

WHAT IS AN ALGORITHM?

The concept of understanding and writing computer algorithms is very essential to understanding the task of programming and every computing student has to imbibe the concepts of algorithms. In fact, algorithms are the basic key to understanding the theory and practice of computing.

Algorithms was named after the 9th century Persian mathematician Al-Khowarizmi. He wrote a treatise in Arabic in 825 AD, *On Calculation with Hindu Numerals*. It was translated into Latin in the 12th century as *Algoritmi de numero Indorum*, which "[Book by] Algoritmus on the numbers of the Indians", where "Algoritmi" was the translator's rendition of the author's name.

An algorithm can be defined as a finite set of steps, which has to be followed while carrying out a particular problem. It is nothing but a process of executing actions step by step.

An algorithm is a distinct computational procedure that takes input as a set of values and results in the output as a set of values by solving the problem. More precisely, an algorithm is correct, if, for each input instance, it gets the correct output and gets terminated.

An algorithm unravels the computational problems to output the desired result. An algorithm can be described by incorporating a natural language such as English, Computer language, or a hardware language.

Characteristics of Algorithms

The main Characteristics or features of Algorithms are;

- **Input:** It should externally supply zero or more quantities or data.
- **Output:** It results in at least one quantity or result.
- **Definiteness:** Each instruction should be clear and unambiguous.
- **Finiteness:** An algorithm should terminate after executing a finite number of steps.
- **Effectiveness:** Every instruction should be fundamental to be carried out, in principle, by a person using only pen and paper.
- **Feasible:** It must be feasible enough to produce each instruction.
- **Flexibility:** It must be flexible enough to carry out desired changes with no efforts.
- **Efficient:** The term efficiency is measured in terms of time and space required by an algorithm to implement. Thus, an algorithm must ensure that it takes little time and less memory space meeting the acceptable limit of development time.
- **Independent:** An algorithm must be language independent, which means that it should mainly focus on the input and the procedure required to derive the output instead of depending upon the language.

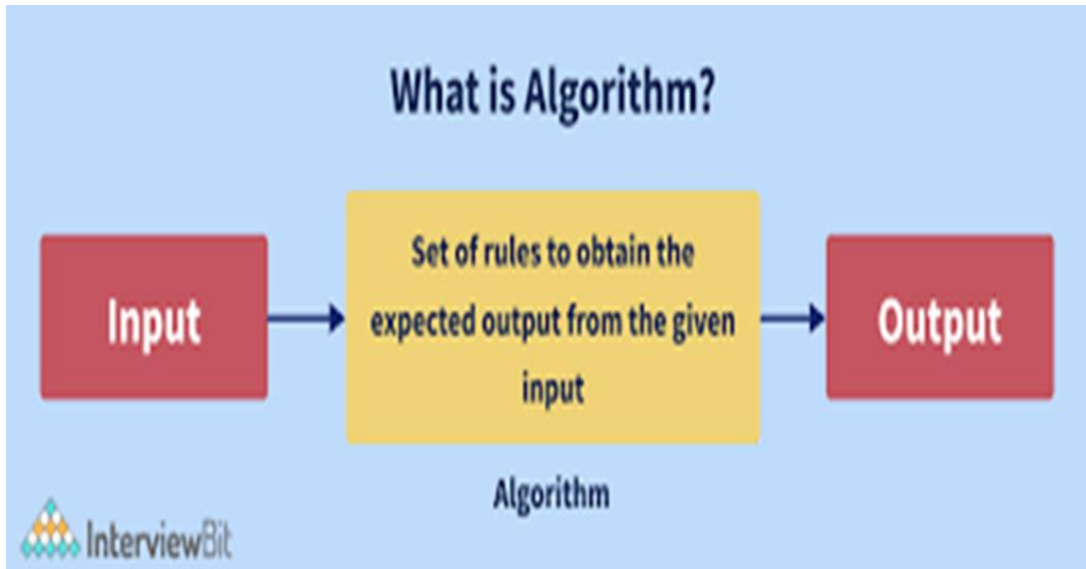
Advantages of Algorithms

- **Effective Communication:** Since it is written in a natural language like English, it becomes easy to understand the step-by-step delineation of a solution to any particular problem.
- **Easy Debugging:** A well-designed algorithm facilitates easy debugging to detect the logical errors that occurred inside the program.
- **Easy and Efficient Coding:** An algorithm is nothing but a blueprint of a program that helps develop a program.
- **Independent of Programming Language:** Since it is a language-independent, it can be easily coded by incorporating any high-level language.

CSC 214: Algorithm & Complexity Analysis (2 Units)

Disadvantages of Algorithms

- Developing algorithms for complex problems would be time consuming and difficult to understand.
- It is a challenging task to understand complex logic through algorithms.



Pseudocode

Pseudocode refers to an informal high-level description of the operating principle of a computer program or algorithm. It uses structural conventions of a standard programming language intended for human reading rather than the machine reading.

Advantages of Pseudocode

- It can be quickly transformed into an actual programming language than a flowchart since it is similar to a programming language.
- The layman or user can easily understand it.
- It can be easily modified as compared to flowcharts.
- Its implementation is beneficial for structured, designed elements.
- It can easily detect an error before transforming it into a code.

Disadvantages of Pseudocode

1. Since it does not incorporate any standardized style or format, it can vary from one user or programmer to another.
2. Error possibility is higher while transforming into a code.
3. It may require a tool for extracting out the Pseudocode and facilitate the drawing flowcharts.
4. It does not depict the design.

CSC 214: Algorithm & Complexity Analysis (2 Units)

Difference between Algorithm and Pseudocode

Algorithm	Pseudocode
i. An algorithm is a formal definition with some specific characteristics that describes a process, which could be executed by a Turing-complete computer machine to perform a specific task. Generally, the word "algorithm" can be used to describe any high level task in computer science. ii. An algorithm is simply a problem-solving process, which is used not only in computer science to write a program but also in our day to day life. It is nothing but a series of instructions to solve a problem or get to the problem's solution. It not only helps in simplifying the problem but also to have a better understanding of it. <i>Problem Case/ Example:</i> <i>Suppose there are 60 students in a class. How will you calculate the number of absentees in the class?</i> iii. 1. Count <- 0, absent <- 0, total <- 60 2. REPEAT till all students counted Count <- Count + 1 3. absent <- total – Count 4. Print "Number absent is:" , absent	Pseudocode is an informal and (often rudimentary) human readable description of an algorithm leaving many granular details of it. Writing a pseudocode has no restriction of styles and its only objective is to describe the high level steps of algorithm in a much realistic manner in natural language. However, Pseudocode is a way of writing an algorithm. Programmers can use informal, simple language to write pseudocode without following any strict syntax. It encompasses semi-mathematical statements. <i>Problem Case/ Example:</i> <i>Suppose there are 60 students in a class. How will you calculate the number of absentees in the class?</i> 1. Initialize a variable called Count to zero, absent to zero, total to 60 2. FOR EACH Student PRESENT DO the following: Increase the Count by One 3. Then Subtract Count from total and store the result in absent 4. Display the number of absent students

The Need for Algorithms (Why do we need Algorithms?)

1. To understand the basic idea of the problem.
2. To find an approach to solve the problem.
3. To improve the efficiency of existing techniques.
4. To understand the basic principles of designing the algorithms.
5. To compare the performance of the algorithm with respect to other techniques.
6. It is the best method of description without describing the implementation detail.
7. The Algorithm gives a clear description of requirements and goal of the problem to the designer.
8. A good design can produce a good solution.
9. To understand the flow of the problem.
10. To measure the behaviour (or performance) of the methods in all cases (best cases, worst cases, average cases)

CSC 214: Algorithm & Complexity Analysis (2 Units)

11. With the help of an algorithm, we can also identify the resources (memory, input-output) cycles required by the algorithm.
12. With the help of algorithm, we convert art into a science.
13. To understand the principle of designing.
14. We can measure and analyze the complexity (time and space) of the problems concerning input size without implementing and running it; it will reduce the cost of design.

ANALYSIS AND COMPLEXITY OF ALGORITHMS

Analysis of algorithms helps us to determine how good or how bad they are in terms of speed or time taken and memory or space utilized. Designing good programs is dependent on how good or how bad the algorithm is and the analysis helps us to determine the efficiency of such algorithms.

Analysis of an algorithm is the same thing as estimating the efficiency of the algorithm. There are two fundamental parameters based on which we can analyse the algorithm and they are Space and Time Complexity.

There is also the concept in Time Complexity of estimating the running time of an algorithm and we have the Best-case, Average-case and Worst-case

OBJECTIVES

- Understand runtime and space analysis or complexity of algorithms
- Know the different types of analysis
- Understand the typical complexities of an algorithm
- Learn how to approximate the time taken by an algorithm

Analysis of Algorithm

The analysis is a process of estimating the efficiency of an algorithm and that is, trying to know how good or how bad an algorithm could be. There are two main parameters based on which we can analyze the algorithm:

Space Complexity: The space complexity can be understood as the amount of space required by an algorithm to run to completion.

Time Complexity: Time complexity is a function of input size n that refers to the amount of time needed by an algorithm to run to completion.

Example. Suppose there is a problem to solve in Computer Science, and in general, we solve a problem by writing a program. If you want to write a program in some programming language like C, then before writing a program, it is necessary to write a blueprint in an informal language. Or in other words, you should describe what you want to include in your code in an English-like language for it to be more readable and understandable before implementing it, which is nothing but the concept of Algorithm.

CSC 214: Algorithm & Complexity Analysis (2 Units)

In general, if there is a problem **P1**, then it may have many solutions, such that each of these solutions is regarded as an algorithm. So, there may be many algorithms such as **A1, A2, A3, ..., An**.

Before you implement any algorithm as a program, it is better to find out which among these algorithms are good in terms of time and memory. It would be best to analyze every algorithm in terms of **Time** that relates to which one could execute faster and **Memory or Space** corresponding to which one will take less memory.

So, the Design and Analysis of Algorithm talks about how to design various algorithms and how to analyze them. After designing and analysing, choose the best algorithm that takes the least time and the least memory and then implement it as a program in C or any preferable language.

We will be looking more on time rather than space because time is a more limiting parameter in terms of the hardware. It is not easy to take a computer and change its speed. So, if we are running an algorithm on a particular platform, we are more or less stuck with the performance that platform can give us in terms of speed.

However, on the other hand, memory is relatively more flexible. We can increase the memory as when required by simply adding a memory card. So, we will focus on time than that of the space. The running time is measured in terms of a particular piece of hardware, not a robust measure. When we run the same algorithm on a different computer which might be faster or use different programming languages which may be designed to compile faster, we will find out that the same algorithm takes a different time.



CSC 214: Algorithm & Complexity Analysis (2 Units)

Types of Time Complexity Analysis

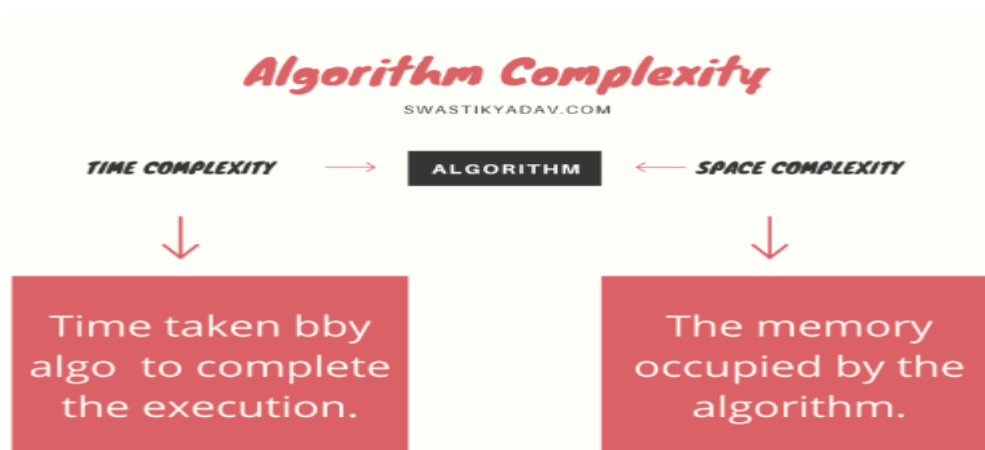
We have three types of analysis related to time complexity, which are:

1. **Worst-case time complexity:** For 'n' input size, the worst-case time complexity can be defined as the maximum amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the maximum number of steps performed on an instance having an input size of n. Computer Scientists are more interested in this.
2. **Average case time complexity:** For 'n' input size, the average case time complexity can be defined as the average amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the average number of steps performed on an instance having an input size of n.
3. **Best case time complexity:** For 'n' input size, the best-case time complexity can be defined as the minimum amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the minimum number of steps performed on an instance having an input size of n.

Complexity of Algorithms

The term algorithm complexity measures how many steps are required by the algorithm to solve the given problem. It evaluates the order of count of operations executed by an algorithm as a function of input data size. To assess the complexity, the order (approximation) of the count of operation is always considered instead of counting the exact steps. $O(f)$ notation represents the complexity of an algorithm, which is also termed as an Asymptotic notation or "**Big O**" notation. Here the f corresponds to the function whose size is the same as that of the input data. The complexity of the asymptotic computation $O(f)$ determines in which order the resources such as CPU time, memory, etc. are consumed by the algorithm that is articulated as a function of the size of the input data.

The complexity can be found in any form such as constant, logarithmic, linear, $n \cdot \log(n)$, quadratic, cubic, exponential, etc. It is nothing but the order of constant, logarithmic, linear and so on, the number of steps encountered for the completion of a particular algorithm. To make it even more precise, we often call the complexity of an algorithm as "running time".



CSC 214: Algorithm & Complexity Analysis (2 Units)

Typical Complexities of an Algorithm

We take a look at the different types of complexities of an algorithm and one or more of our algorithm or program will fall into any of the following categories;

a) Constant Complexity

Imposes a complexity of $O(1)$. It undergoes an execution of a constant number of steps like 1, 5, 10, etc. for solving a given problem. The count of operations is independent of the input data size.

b) Logarithmic Complexity

Imposes a complexity of $O(\log(N))$. It undergoes the execution of the order of $\log(N)$ steps. To perform operations on N elements, it often takes the logarithmic base as 2.

For $N = 1,000,000$, an algorithm that has a complexity of $O(\log(N))$ would undergo 20 steps (with a constant precision). Here, the logarithmic base does not hold a necessary consequence for the operation count order, so it is usually omitted

c) Linear Complexity

Imposes a complexity of $O(N)$. It encompasses the same number of steps as that of the total number of elements to implement an operation on N elements. For example, if there exist 500 elements, then it will take about 500 steps. Basically, in linear complexity, the number of elements linearly depends on the number of steps. For example, the number of steps for N elements can be $N/2$ or $3*N$.

It also imposes a run time of $O(n*\log(n))$. It undergoes the execution of the order $N*\log(N)$ on N number of elements to solve the given problem.

For a given 1000 elements, the linear complexity will execute 10,000 steps for solving a given problem.

d) Quadratic Complexity

It imposes a complexity of $O(n^2)$. For N input data size, it undergoes the order of N^2 count of operations on N number of elements for solving a given problem.

If $N = 100$, it will endure 10,000 steps. In other words, whenever the order of operation tends to have a quadratic relation with the input data size, it results in quadratic complexity.

For example, for N number of elements, the steps are found to be in the order of $3*N^2/2$.

e) Cubic Complexity

It imposes a complexity of $O(n^3)$. For N input data size, it executes the order of N^3 steps on N elements to solve a given problem.

For example, if there exist 100 elements, it is going to execute 1,000,000 steps.

f) Exponential Complexity

It imposes a complexity of $O(2^n)$, $O(N!)$, $O(n^k)$, For N elements, it will execute the order of count of operations that is exponentially dependable on the input data size.

For example, if $N = 10$, then the exponential function 2^N will result in 1024. Similarly, if $N = 20$, it will result in 1048 576, and if $N = 100$, it will result in a number having 30 digits.

The exponential function $N!$ grows even faster; for example, if $N = 5$ will result in 120. Likewise, if $N = 10$, it will result in 3,628,800 and so on.

Since the constants do not hold a significant effect on the order of count of operation, so it is better to ignore them.

CSC 214: Algorithm & Complexity Analysis (2 Units)

Thus, to consider an algorithm to be linear and equally efficient, it must undergo N , $N/2$ or $3*N$ count of operation, respectively, on the same number of elements to solve a particular problem.

A summary of these complexities is given below:

increasing order of complexity ↓		Big O	Remarks
	Constant	$O(1)$	not affected by input size n
	Logarithmic	$O(\log n)$	e.g. binary search
	Linear	$O(n)$	proportional to input size n
	Linearithmic	$O(n \log n)$	e.g. merge sort, heap sort
	Polynomial	$O(n^k)$	k is some constant, e.g., $k = 1$ is linear, $k = 2$ is quadratic, $k = 3$ is cubic
	Exponential	$O(k^n)$	k is some constant
	Factorial	$O(n!)$	within the exponential family